Protocolo de comparação justa: ILS vs VNS-SOP (SOP e SOPMV)

Resumo: a comparação justa se faz por **orçamento de tempo de CPU igual**, não por iterações. Cada algoritmo deve rodar várias vezes por instância, com tuning equivalente e testes estatísticos, que é o padrão da literatura. O protocolo abaixo se baseia no artigo do SA-SOPMV (Mathematics 2024, 12(19), 3089, doi 10.3390/math12193089).

1. O que o artigo faz (e o que dá para copiar)

- **Critério de parada por tempo:** o SA usa Maxtime = 1.5 × Nset segundos, onde Nset é o número de conjuntos (cerca de 20% dos nós). Para a 217vm1084 isso dá ~325 s. Não há limite de iterações.
- **Calibração:** Taguchi L16 em 4 instâncias, 10 execuções por configuração, medida por ARPD. O Maxtime foi o parâmetro mais influente, então o orçamento de tempo domina o resultado.
- **Métricas:** RPD em relação ao melhor valor conhecido, (P_best − P_alg)/P_best × 100, com média sobre instâncias (ARPD). Execuções sem solução viável recebem RPD = 100%. Nas tabelas de SOP aparecem melhor valor e média por instância.
- **Instâncias:** conjuntos pequeno (20) e grande (47 × g1/g2), derivados de Archetti e Pěnička. Para SOPMV: 10%, 20% e 30% de conjuntos obrigatórios, $\omega \in \{0.4, 0.6, 0.8\}$ e perfis de lucro g1/g2.

Trocar SA por ILS é viável, com quatro ressalvas

1. As instâncias SOPMV não são públicas (só sob demanda aos autores). Peça a eles ou gere as suas com semente fixa e publique o gerador.
2. O artigo só compara o SA com o VNS-SOP em SOP puro (0% de obrigatórias). No SOPMV, ele compara com MILP. Seu "VNS-SOPMV" é, portanto, uma adaptação sua, que deve ser documentada.
3. O artigo copia os resultados do VNS de outro trabalho e outra máquina, e admite que o SA gasta mais tempo. Isso é justamente o que se quer evitar: **reexecute o VNS na sua máquina**.
4. O SOPMV do artigo usa o melhor de 5 execuções. Reporte também média e desvio.

2. Protocolo recomendado

2.1 Orçamento: tempo de CPU como critério principal

Uma iteração do ILS (perturbação + busca local) e uma do VNS (shaking + busca local + troca de vizinhança) custam coisas diferentes, então iterações não são comparáveis. Defina:

- Parada **somente por tempo** nos dois (num-iterations e num-iterations-unimproved do VNS em valores enormes; nenhum critério de estagnação no ILS).
- Orçamento proporcional ao tamanho, T = c × Nset com c = 1.5 (valor do artigo), em vez de 600 s fixos. Para a Berlin52, 600 s é um exagero, e para instâncias de ~1000 nós pode ser pouco.
- Se possível, rodar em 2–3 níveis (0.5×, 1×, 2× o orçamento) para mostrar se um método vence só com tempo curto ou longo.
- Como complemento independente de máquina, instrumentar os dois códigos para contar avaliações de rota (inserções/custos calculados) e reportar como informação extra.

2.2 Mesmas condições de execução

Mesma máquina, uma thread por processo, com taskset e sem outra carga. Mesmo compilador e flags (-O3). Matriz de distâncias pré-computada nos dois, com distância euclidiana arredondada para cima como no artigo. O tempo inclui a construção da solução inicial e exclui a leitura do arquivo. Mesmo critério de viabilidade das obrigatórias nos dois (restrição rígida ou penalidade, o mesmo em ambos).

2.3 Repetições e sementes

Mínimo de 10 execuções por instância e algoritmo; 20–30 se possível, pelo ganho de poder dos testes. Cada execução usa uma semente distinta e registrada (ex.: 202601 + i para o ILS). O VNS não expõe semente: altere a linha que usa o relógio para receber um parâmetro, ou ao menos registre a semente gerada. Uma execução com semente fixa é uma amostra de tamanho 1 e não sustenta conclusão alguma.

2.4 Tuning equivalente e fora do conjunto de teste

Os parâmetros padrão do VNS vêm dos autores e já foram calibrados. Dê ao ILS esforço comparável de calibração (irace, ou DOE como o Taguchi do artigo), em instâncias **separadas** das de teste. Um limite de iterações sem melhora só entra no ILS se a calibração o justificar.

2.5 Instâncias

A Berlin52 não discrimina: o SA do artigo resolve todas as variantes ótimas em ~2 s. Use uma amostra estratificada: pequenas (~50–100 nós), médias (~200–400) e grandes (>500, até a 217vm1084). Cruze com MV $\in$ {0%, 10%, 20%, 30%}, $\omega \in$ {0.4, 0.6, 0.8} e g1/g2. O 0% serve de ponte com a literatura, porque permite comparar com os números publicados do VNS e do MASOP. Instâncias SOPMV inviáveis (comuns com ω baixo e MV alto) devem ser separadas e relatadas à parte.

2.6 Métricas

- Melhor, média, desvio e pior por instância.

- RPD contra o BKS. No SOP, use os melhores valores publicados. No SOPMV, use o melhor encontrado entre todas as execuções de todos os algoritmos, e o ótimo do Gurobi nas instâncias pequenas.
- Taxa de execuções inviáveis.
- Curva de convergência: melhor valor em 10%, 25%, 50% e 100% do tempo, e tempo até o melhor valor.

2.7 Estatística

- Wilcoxon pareado sobre a média por instância.
- Por instância, Mann–Whitney sobre as 10 execuções, com correção de Holm.
- Tamanho de efeito (Vargha–Delaney A12) e contagem de vitórias/empates/derrotas.
- Com um terceiro método, Friedman com pós-teste.

3. O que corrigir no teste atual

Hoje	Problema	Ajuste
2000 it. / 1000 sem melhora	Unidades diferentes entre ILS e VNS	Só tempo, escalado por Nset
1 execução, semente ILS fixa, VNS no relógio	Sem variância, sem reprodutibilidade	≥10 execuções, sementes registradas
Berlin52	Trivial, todos empatam no ótimo	Amostra estratificada até ~1000 nós
Guloso "de cada código"	A solução inicial pode favorecer um lado	Mesma solução inicial nos dois, ou reportar o custo dela

4. Próximos passos

Antes de rodar tudo, fazer um piloto em ~6 instâncias com 3 repetições para medir o tempo total e ajustar o escopo. O custo cresce rápido: instâncias × variantes × 10 execuções × 2 algoritmos × 1.5·Nset segundos. Com uma execução por núcleo, isso é viável. Ainda a fazer: script de execução (taskset, sementes, logs) e script de análise em Python (RPD, Wilcoxon, Holm, curvas de convergência).